

Hur C-strängar fungerar

Ser man på hur strängar i C fungerar kan man säga att det egentligen är en *nollterminerad vektor* av `char`. Med nollterminerad menas att varje sträng bör sluta med värdet 0. Här avses inte siffran '0', utan ASCII-värdet 0, som inte är ett giltigt tecken som kan visas på skärmen. Detta tecken anger var strängen slutar, och de flesta funktioner som använder sig av C-strängar är beroende av att det finns på korrekt plats.

Figur A-1. Representation av C-sträng

H	e	l	l	o	!	\0
---	---	---	---	---	---	----

Om inte ett tecken 0 terminerar strängen kan allvarliga fel ske då funktioner som blint väntar sig ett 0 inte hittar det utan fortsätter ut i okänt minne. Sådana fel är inte helt sällsynta. C-strängar är därför alltid ett tecken längre än vad antalet egentliga tecken i strängen kräver. I bilden ovan finns det sex tecken som kan visas på t.ex. skärmen, men strängen är ändå sju tecken lång för att rymma en sista 0.

Skapa C-strängar

Eftersom C-strängar är helt enkelt vektorer med element av typen `char` kan man skapa strängar precis som `char`-vektorer, t.ex.:

```
char Hello1[] = "Hello!";  
char * Hello2 = "Hello again!";  
char Hello3[5];  
  
Hello3[0] = 'H';  
Hello3[1] = 'e';  
Hello3[2] = 'j';  
Hello3[3] = '!';  
Hello3[4] = '\0';
```

Den första variabeln `Hello1` definierar en vektor utan storlek, och låter kompilatorn räkna ut denna. Strängen blir exakt så lång som det behövs för alla tecken och terminatorn 0, d.v.s. sju tecken. `hello2` däremot använder en alternativ syntax, men som åstadkommer exakt samma resultat. Med `hello3` skapas en vektor med en explicit storlek. Vi fyller sedan manuellt vektorn genom att ge värden åt de enskilda tecknen, utan att glömma det sista tecknet `'\0'`. Då man definierar en explicit storlek måste man vara mycket noggrann så att den tilltänkta strängen rymms i vektorn.

Kan man naturligtvis även dynamiskt allokera C-strängar. Man allokerar då en vektor av datatypen `char` med den önskade storleken.

Manipulera enskilda tecken

Man kan manipulera de enskilda tecknen i C-strängar precis som element av typen `char` i vilken vektor som helst. I exemplet ovan tilldelades en sträng ett värde genom elementvis tilldelning. Man kan accessera elementen normalt genom operatoren `[]` eller med hjälp av pekararitmetik. Vi kunde skriva om ovanstående tilldelning till följande:

```
char Hello3[5];

*(Hello3 + 0) = 'H';
*(Hello3 + 1) = 'e';
*(Hello3 + 2) = 'j';
*(Hello3 + 3) = '!';
*(Hello3 + 4) = '\\0';
```

I vissa fall är det lämpligt att använda pekararitmetik, men i de flesta fall är det lättare och snyggare att använda normal vektorindexering med []. Det är ingen skillnad ur effektivitetssynpunkt, eftersom kompilatorn översätter allting till pekararitmetik innan den egentliga koden genereras. Använd den syntax som verkar lättare och som ger mindre fel.

[Förutgående](#)

C-strängar

[Hem](#)[Upp](#)[Nästa](#)

Tilldelning och jämförelse